

系统设计说明书

本文描述当前代码中的 Agent 管理控制面：定位、分层、数据模型、访问控制、Agent 运行时、工具与沙箱、评测与 A/B 实验、观测与部署边界。实现以仓库为准；演进项单独标注，不与现状混写。

文档版本	1.0 · 对应应用 0.1.0
日期	2026-09-09
范围	agentforge-control-plane 全模块
读者	架构、后端、安全、运维评审
状态	基于现网代码的设计说明，供评审与交接

目录

1. 1. 文档说明与目标
2. 1.1 要解决的问题 · 1.2 非目标 · 1.3 设计原则
3. 2. 总体架构
4. 2.1 逻辑视图 · 2.2 技术栈 · 2.3 模块地图 · 2.4 请求与数据流
5. 3. 功能架构
6. 4. 数据模型
7. 5. 身份、租户与权限
8. 6. Agent 运行时（调试台入口）
9. 7. 能力注册：Skill / MCP / 浏览器 / 沙箱
10. 8. 工作空间与检查点
11. 9. 质量：数据集、评测、A/B 实验
12. 10. 观测
13. 11. 前端管理台
14. 12. 隔离与安全边界
15. 13. 部署、配置与运维
16. 14. 接口一览
17. 15. 演进建议

1. 文档说明与目标

AgentForge Control Plane 是本地可运行的 企业 Agent 管理控制面，不是又一个聊天机器人。FastAPI 同时提供 REST 与静态管理台；业务数据默认落 MySQL（开发测试可用 SQLite）。
管理台地址：`http://127.0.0.1:8080`，OpenAPI 在 `/docs`。

1.1 要解决的问题

- 谁能看、谁能改：租户 + 角色权限；默认属主可见，同租户可按角色共享，跨租户不可见。
- Agent 用什么干活：模型、Skill 规程、MCP / 浏览器工具、沙箱策略的集中登记与绑定。
- 对话能不能复盘：会话、消息、本地 Trace，可选推送到 AgentScope Studio / Langfuse。
- 质量靠什么证明：数据集回归、在线抽检 / 离线评测、A/B 分流实验。

1.2 非目标（当前版本不做）

- 对外多租户 SaaS 的物理隔离（独立库、独立 VPC、独立 K8s Namespace）。
- 把 Agent 推理进程做成每租户独立运行时或服务网格。
- 完整 RAG 知识库产品（本仓库内置的 `search\_knowledge` 只检索已登记 Skill 正文）。
- 工作流编排的生产级执行引擎（`Workflow.graph` 可保存 DAG，执行校验与调度尚未作为主路径）。

1.3 设计原则

1. 控制面与模型解耦：聊天走 OpenAI 兼容 `/chat/completions`，未安装 AgentScope 也能跑通控制面。
2. 绑定才生效：Skill / MCP / 沙箱必须挂到 Agent；未绑定的工具名在运行时拒绝。
3. 失败可续跑：调试台把进行中状态写入 workspace checkpoint，支持 `/playground/resume`。
4. 没有密钥不假装已调用：缺凭证时走预览回复，并在链路里标明 preview。
5. 租户 ID 不信任模型：工具侧使用服务端 `ExecutionContext`，模型参数里的 tenant_id 不作授权依据。
6. 沙箱默认不回退宿主机：Docker 不可用时拒绝在控制面主机执行用户代码。

2. 总体架构

2.1 逻辑视图



2.2 技术栈

层	选型	说明
API	FastAPI 0.115+ / Uvicorn / Python 3.11+	同源托管静态页，无需单独 Node 构建
前端	static/ 单页 (app.js)	按页面拉 API 渲染，无独立前端工程
ORM	SQLAlchemy 2.x	启动时 create_all + ensure_schema 补列
数据库	MySQL 8 utf8mb4；测试用 SQLite	DATABASE_URL 配置
模型	OpenAI 兼容 HTTP	httpx 同步调用，超时 45s
工具	MCP Streamable HTTP / 内置工具 / Playwright	浏览器默认 headless Chromium
沙箱	加固 Docker；本地 python -I 默认关闭	镜像白名单，禁止自动 pull
观测	库内 Trace JSON + Studio OTLP + 可选 Langfuse	Studio 默认 127.0.0.1:3000

2.3 模块地图

路径	职责
app/main.py	生命周期、资源 CRUD、仪表盘、会话、调试台主循环
app/routers/auth.py	登录登出、当前用户、租户与用户管理

app/routers/evaluations.py	数据集、导入、评测任务与报告导出
app/routers/experiments.py	A/B 实验 CRUD、启停、分流、对比
app/access/	令牌、权限点、租户行级过滤、分享策略
app/services/agentscope_adapter.py	AgentScope 初始化、模型名归一、complete_chat
app/services/tool_runtime.py	Skill 注入、工具白名单、内置/远程工具执行
app/services/mcp_stream.py	Streamable HTTP 探测与调工具
app/services/browser_runtime.py	Playwright 会话，按 scope_key 隔离 Context
app/services/sandbox_runtime.py / docker_sandbox.py	策略选择后端、Docker 加固执行
app/services/agent_workspace.py	每 Agent 工作目录、会话文件、checkpoint
app/services/execution_context.py	服务端可信租户/Agent 上下文
app/services/eval_runner.py	离线评测工人（进程内线程轮询）
app/services/experiment_runtime.py	粘滞分流与实验指标
app/services/studio_tracer.py / langfuse_tracer.py	外部轨迹导出
static/	管理台 UI
skills/	Skill 源文件（SKILL.md）
workspaces/	会话、链路、沙箱工作目录、数据集文件

2.4 请求与数据流

正常 Agent 入口是调试台，而不是独立的推理集群。

评测任务复用同一套 ``generate_chat_reply``，因此 Skill、MCP、沙箱在回归里同样生效。离线评测写入 ``evaluation_runs.status=queued``，由启动时拉起的 daemon 线程领取。

3. 功能架构

中心	模块	能力
运行中心	运行概览	租户内会话量、成功率、延迟、Token；Agent 健康度与最近会话。部分数字有展示放大，列表为真实记录。
会话查询	按 Agent / 状态 / 关键字检索 Conversation，查看消息与 Trace	
AgentScope Studio	内嵌或外链 Studio，看步骤与 Token；Studio 关闭不影响对话	
数据测试	数据集、在线抽检（最多 10 条同步）、离线回归、CSV 导出、重跑	
Agent 调试台	每 Agent 工作空间对话；可选实验分流；失败续跑	
构建中心	Agent 管理	人设、模型、版本、Skill / MCP / 沙箱绑定、复制重命名
MCP 工具	HTTP Stream / SSE / StdIO / 内置本地工具与浏览器工具；探测发现	
Skill 管理	Markdown 规程，可写回 skills/*/SKILL.md	

平台设置	模型配置	供应商、model_id、base_url、温度、密钥或环境变量名
沙箱管理	镜像、CPU/内存/超时/网络；试跑 print('sandbox-ok', 1+1)	
权限管理	租户、角色权限点、用户启用状态	
质量	A/B 实验	多变体权重、粘滞策略、holdout、对比延迟与失败率

4. 数据模型

绝大多数业务表混入 `TenantOwnedMixin`：`tenant\_id`、`owner\_id`。启动时若缺列会 ALTER 补上，并把空 `tenant\_id` 填成默认租户。

表	关键字段	用途
tenants	slug, name, status	租户
users / roles / auth_tokens	password_hash, permissions JSON, token	账号与会话
agents	system_prompt, skill_ids, mcp_ids, sandbox_id, workspace	Agent 注册与绑定
model_configs	provider, model_id, base_url, api_key / api_key_ref	推理配置；接口脱敏
skills	instruction, source, version	提示词模块
mcp_servers	transport, endpoint, config.tools	工具服务
sandbox_policies	runtime, cpu/memory, timeout,	代码执行策略
workflows	network_mode graph JSON	编排草稿
conversations / chat_messages	session_id, latency, tokens	会话与消息
traces	spans JSON, langfuse_url	本地链路
datasets / dataset_cases	kind, input, expected, tags	评测集
evaluation_runs / evaluation_results	scorer, status, actual, reason	评测任务与逐条结果
experiments / variants / assignments / events	strategy, weight, unit_key, holdout	A/B

标识与唯一约束

5. 身份、租户与权限

5.1 认证

- 登录：`POST /api/auth/login`，PBKDF2-HMAC-SHA256（约 12 万轮），签发随机 Bearer，TTL 7 天，落 `auth\_tokens`。
- 请求：`Authorization: Bearer`；平台管理员可用 `X-Tenant-Id` 切换工作租户，普通用户禁止。
- 权限：角色 JSON 权限点；`\*` 与 `platform:admin` 视为全权限。

5.2 授权模型

对齐 AgentScope「默认属主可见」：

1. 查询先按 `tenant\_id == 当前租户 OR tenant\_id IS NULL` 过滤。
2. 属主（`owner\_id` 为空或等于自己）可见可写（还需具备 write 权限点）。

- 3. `TenantSharePolicy`：同租户且具备对应 read/write 的成员，可看见（及编辑）他人资源。
- 4. 跨租户：`get\_row` 返回 404，不暴露存在性。

权限点	分组
* / platform:admin / tenant:admin	平台与租户
user:* / role:*	IAM
agent / mcp / skill / model / workflow / sandbox 的 read、write	构建
session:read write、trace:read	运行
eval:read run、experiment:read write	质量

种子角色大致为：平台管理员（全权限）、开发者（构建+评测）、审计员（会话与链路只读）、演示账号。

5.3 跨资源引用

Agent 绑定 Skill / MCP / Sandbox 时，`validate\_agent\_bindings` 要求 ID 必须属于同一 `tenant\_id`，否则 422。运行时 `selected\_skills / selected\_mcps / selected\_sandbox` 再滤一遍，历史脏绑定会被忽略。

6. Agent 运行时（调试台入口）

主入口：`POST /api/playground/run` 与 `POST /api/playground/resume`。需要 `session:write` 与 `agent:write`。

6.1 准入

- 1. 解析 Agent、模型配置；模型 disabled 则 409。
- 2. 若带 `experiment\_id` 且实验为 running，按策略分流，可能改写实际 Agent；holdout 仍用所选 Agent。
- 3. 会话不得跨租户、不得跨 Agent 工作空间（实验场景会新开会话）。
- 4. 历史优先读 workspace 会话文件，否则回落 `chat\_messages`。

6.2 generate_chat_reply

- 1. 创建 `ExecutionContext(tenant\_id, agent\_id, execution\_id)` 写入 ContextVar。
- 2. 无密钥：预览回复；时间类问题若绑定了 `get\_current\_time` 仍可调工具。mode=`preview`。
- 3. 有密钥：拼系统提示（人设 + 已绑 Skill 正文 + 工具提示）；组装 OpenAI tools（MCP + 可选沙箱 spec）。
- 4. 循环调用 `complete\_chat`（最多 4 轮；含 browser_ 工具时 8 轮）。模型返回 tool_calls 则在控制面执行，结果以 role=tool 回灌。
- 5. 未授权工具名直接返回错误 JSON，不执行。
- 6. 异常写入 checkpoint（status=failed），回复中带错误摘要（密钥已脱敏）。
- 7. finally：关闭本轮 browser scope，复位 ExecutionContext。

6.3 返回前落盘

`\_finalize\_playground` 同步完成：

- 写 ChatMessage、更新或创建 Conversation、写 Trace.spans。
- `persist\_run` 写入 Agent workspace 的 sessions / traces JSON。
- 同步 HTTP 导出 AgentScope Studio（短超时，失败不影响主流程语义但仍占用返回路径）。
- 若走实验，记 ExperimentEvent。

运行时形态

7. 能力注册：Skill / MCP / 浏览器 / 沙箱

7.1 Skill

Skill 是给模型看的 Markdown 规程，不是可执行包。来源可以是库内 `instruction` 或仓库 `skills/\*/SKILL.md`。绑定后由 `skill\_prompt\_block` 拼进系统提示。

7.2 MCP

传输	行为
HTTP Stream (推荐)	Streamable HTTP 探测、列出 tools、运行时 call_tool
SSE / StdIO	登记与展示；远程调用以 HTTP Stream 为主路径
内置本地工具	get_current_time、calculate、search_knowledge、list_agents
内置浏览器	browser_open / text / links / close

远程工具若 schema 含 tenant_id 字段，由服务端覆盖为当前 Agent 租户，不采用模型传入值。

7.3 浏览器

- 进程内一个 Playwright Chromium；每个 ExecutionContext.scope_key 一个 BrowserContext + Page。
- 全局锁串行化操作，避免并发打到同一驱动。
- 默认拒绝 localhost 与非全球单播地址；`BROWSER\_ALLOW\_PRIVATE\_NETWORK=1` 才放开。
- 仅 http/https，禁止 URL 内嵌用户名密码。

7.4 沙箱

Agent 未绑沙箱或策略停用时，`sandbox\_run\_python/shell` 被拒绝。

后端	选用条件	隔离
docker	runtime 解析为 docker 镜像且在白名单	一次性容器：--rm、read-only 根盘、cap-drop ALL、no-new-privileges、pids/memory/cpu/ ulimit、默认 --network none；工作目录 bind-mount。Docker
agentscope_runtime	runtime 名称像官方镜像且包可用	歪哥男魁梓麟；歪殿再寝惠翠地
local	仅当 ALLOW_UNSAFE_LOCAL_SANDBOX=1	python -l 或 /bin/sh，可选 socket 阻断与内存 preexec。生产默认关闭。

联网沙箱必须配置 `SANDBOX\_EGRESS\_NETWORK`。镜像禁止 `--pull` 自动拉取。

8. 工作空间与检查点

根目录：环境变量 `WORKSPACES\_DIR`，默认仓库下 `workspaces/`。

```
workspaces/tenants/{tenant_id}/agents/{agent_id}-{slug}/
  sessions/{session_id}.json
  traces/{trace_id}.json
  files/
  manifest.json
```

沙箱工作目录：`workspaces/\_sandboxes/tenant-{id}/policy-{policyId}-{name}/{execution\_id}/`。checkpoint 记录 run_id、status、next (llm|tool)、working 消息、pending_tools、已完成 tool id、usage、error。resume 仅允许 status 为 failed 或 running 的检查点。

9. 质量：数据集、评测、A/B 实验

9.1 数据集

支持手工用例与导入 CSV / JSON / JSONL (最大约 5MB、2000 条)。字段别名覆盖中英文 (问题 / input、期望 / expected 等)。导入 `on\_duplicate` 默认 skip。文件副本落 `workspaces/\_datasets/{id}/`。

9.2 评测

项	说明
在线	同步，最多 10 条
离线	queued → 后台线程每约 1.2s 领取一条 running
打分	contains / exact / regex / llm (另选裁判模型)
执行	每条用例调用 generate_chat_reply，与调试台同绑定
操作	resume (跳过已完成 case)、重跑、取消、导出 CSV

领取任务目前无数据库行锁，多进程部署会存在重复领取风险。见第 15 章。

9.3 A/B 实验

策略	粘滞单位
user_hash	同一用户哈希到同一变体
session_hash	同一会话同一变体
user_first	用户首次按权重随机，之后粘滞
random	每次请求独立抽，不粘滞

2-6 个变体，权重分流；traffic_percent 可留 holdout。调试台可挂实验。对比接口汇总延迟、Token、状态。

10. 观测

- 本地 Trace：每次 playground / 评测写 spans (接收消息、解析 Agent、注入 Skill、关联工具、模型调用、工具结果、回复)。
- AgentScope Studio：登记 run 并推 OTLP；环境变量 `AGENTSCOPE\_STUDIO\_URL`。
- Langfuse：配置公钥/密钥/Host 后导出；缺失则仅本地。
- 健康检查：`GET /api/health`、`/api/observability`。

生产建议：Trace 明细进 OTLP 后端，数据库只留索引与业务标签 (见 ARCHITECTURE.md)。

11. 前端管理台

单页应用：侧栏按权限隐藏入口。调试台为会话气泡 + 右侧执行链路；输入栏固定在对话区底部，消息列表单独滚动。静态资源以 query 版本号刷新缓存。登录态存 localStorage 令牌。

12. 隔离与安全边界

结论先说清楚

对象	现状	不是什么
数据	tenant_id 行级过滤 + 属主/同租户分享	独立数据库、独立 schema、数据库 RLS
磁盘	路径含 tenants/{id}	强制独立卷、独立 OS 用户、强制 chroot

Agent 推理	控制面内同步 HTTP	每 Agent / 每租户独立进程或 VM
浏览器	Playwright Context + 全局锁	独立浏览器集群
代码执行	短命加固容器 (若 Docker 可用)	每租户独立 Docker daemon / K8s Namespace

纵深防御已做：绑定双校验、ExecutionContext、沙箱不回退宿主机、浏览器禁私网、密钥接口脱敏。部署层的分库、IAM、NetworkPolicy 仍需另行建设。

逻辑隔离缺口：`tenant\_id IS NULL` 对所有租户可见；部分 name 全局唯一；平台管理员可切换租户。

13. 部署、配置与运维

13.1 运行

```
.venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8080 --reload
```

启动：建表补列、空 MySQL 可从 SQLite 拷贝、种子 IAM 与演示数据、清理早期展示会话、初始化 AgentScope、启动 eval-worker。

13.2 关键环境变量

变量	作用
DATABASE_URL	mysql+pymysql://... 或 sqlite
DASHSCOPE_API_KEY / OPENAI_API_KEY	模型密钥 (也可写在模型配置)
AGENTSCOPE_STUDIO_URL	Studio 地址
LANGFUSE_*	可选轨迹
WORKSPACES_DIR / SANDBOXES_DIR	工作区与沙箱根
SANDBOX_ALLOWED_IMAGES / SANDBOX_EGRESS_NETWORK / SANDBOX_DEFAULT_IMAGE	Docker 白名单与出网
ALLOW_UNSAFE_LOCAL_SANDBOX	允许宿主机本地沙箱 (仅开发)
BROWSER_HEADED / BROWSER_ALLOW_PRIVATE_NETWORK	有头浏览器 / 允许私网
EVAL_WORKER=0	关闭评测线程

13.3 测试

pytest 使用独立 SQLite 与测试 workspace，覆盖 API、权限、租户绑定、沙箱、评测、实验、MCP、浏览器、checkpoint 等。自动化测试不依赖开发者的 MySQL。

14. 接口一览

分组	方法与路径
认证 IAM	POST /api/auth/login logout ; GET /api/auth/me ; GET/POST /api/tenants ; GET/POST/PUT /api/users
运行	GET /api/health api/observability api/dashboard api/sessions api/sessions/{id} api/traces/{id}

注册表	GET/POST/PUT/DELETE /api/{agents mcp skills models workflows sandboxes roles} ; PATCH .../status ; 复制/重命名 Agent
探测	POST /api/sandboxes mcp skills models/{id}/test
调试台	POST /api/playground/run resume ; GET .../sessions/{session_id} ; GET /api/agents/{id}/workspace
评测	/api/datasets* 导入与用例 ; /api/evaluations 创建、online、resume、run、cancel、export.csv
实验	/api/experiments CRUD ; start/pause/complete/assign/compare

15. 演进建议

以下尚未实现，作为保护 Agent 入口、支撑更高并发时的设计方向。

15.1 热路径与冷路径

用户等待的是这一轮 reply，不要把整次 `generate\_chat\_reply` 丢进任务队列。会话、checkpoint、返回 JSON 必须同步成功；Studio / Langfuse / 仪表盘汇总可入队。

15.2 限流、舱壁、熔断

按租户 QPS、用户 QPS、Agent 在途、模型供应商在途分层限流。playground / eval / sandbox / browser 分舱壁。LLM、单个 MCP、沙箱、Studio 分别熔断；LLM 打开时走现有 checkpoint，不伪装预览成功。

15.3 队列与缓存

Redis：共享令牌桶、熔断状态、配置快照、observe/eval 队列。评测改为真队列 + 状态机，避免多进程抢同一条 queued。缓存 Agent/Skill/MCP 快照，不缓存模型生成；密钥不进 Redis 明文。

15.4 物理隔离（合规需要时）

控制面过滤保留作纵深防御。物理边界在部署：每租户 schema 或实例、独立 workspace 卷、沙箱 Namespace / NetworkPolicy、密钥独立托管。那是拓扑变更，不是再加一个 tenant_id 字段。

15.5 建议落地顺序

- 1. P0：complete_chat 异步化；Studio 导出移出返回路径；本进程限流与 LLM 熔断。
- 2. P1：Redis 共享配额与评测队列；配置 L2 缓存。
- 3. P2：调试台 SSE；配额与熔断可观测；按租户配置限额。

第一刀（若只改一处）